Displacement-Based Frame Sampling

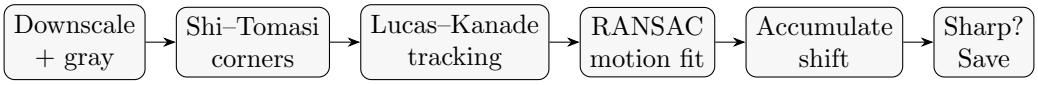
Theory behind `preprocessing/extract_frames.py`

Raphael Pangilinan
COSC 4346 — Introduction to Deep Learning

October 12, 2026

1 Overview

The corn videos are recorded by a moving GoPro, so consecutive frames are nearly identical and labeling every frame would be redundant. The script is a *displacement-based keyframe sampler*: it estimates how far the camera view moves between consecutive frames, accumulates that movement, and saves a frame each time the accumulated shift exceeds a threshold, provided the frame is sharp.



2 Preprocessing

Each frame is resized to 640×360 and converted to grayscale. Motion estimation only needs brightness structure, not color, and the reduced frame has roughly $9\times$ fewer pixels than 1080p. Because the motion measured at the small scale is proportional to motion at full scale, the threshold keeps its meaning as a fraction of the frame width. The saved image is the original full-resolution color frame.

3 Feature Detection: Shi–Tomasi Corners

Not every pixel can be tracked. In a flat region, any displacement looks the same; on an edge, motion is observable only perpendicular to the edge (the *aperture problem*); at a corner, the patch changes under any displacement, so its new position is unambiguous.

For each window, the *structure tensor* is built from the image gradients I_x, I_y:

$$M = \sum_{\text{window}} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}. \tag{1}$$

Its eigenvalues λ_1, λ_2 measure how strongly intensity changes in the two principal directions. Shi and Tomasi [1] score a window by $\min(\lambda_1, \lambda_2)$; a corner has both eigenvalues large. The call `goodFeaturesToTrack(gray, 300, 0.01, 8)` keeps at most 300 corners, each scoring at least 1% of the strongest, spaced at least 8 pixels apart.

4 Tracking: Pyramidal Lucas–Kanade Optical Flow

Lucas–Kanade [2] assumes *brightness constancy*: a point keeps its intensity as it moves, $I(x, y, t) = I(x + u,\, y + v,\, t + 1)$. A first-order Taylor expansion gives the optical flow constraint

$$I_x u + I_y v + I_t = 0, \tag{2}$$

one equation in two unknowns. Assuming all pixels in a small window share the same motion yields an overdetermined system, solved by least squares:

$$\begin{bmatrix} u \\ v \end{bmatrix} = -M^{-1} \sum_{\text{window}} \begin{bmatrix} I_x I_t \\ I_y I_t \end{bmatrix}. \tag{3}$$

The matrix being inverted is the same structure tensor M from the previous section. The solve is stable only when M is well-conditioned, which is exactly what the Shi–Tomasi criterion selects for; the two methods are designed to be used together.

The Taylor approximation holds only for motions of one or two pixels. The pyramidal implementation [3] estimates motion on a coarse, downsampled image first and refines it level by level, allowing displacements of tens of pixels. The returned `status` array marks which points were tracked successfully.

5 Robust Motion Fitting: RANSAC

The tracked point pairs contain outliers: leaves moving in the wind, false matches, and points leaving the frame. `estimateAffinePartial2D` fits a four-degree-of-freedom similarity transform (rotation θ, uniform scale s, translation t_x, t_y):

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s\cos\theta & -s\sin\theta \\ s\sin\theta & s\cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}. \tag{4}$$

RANSAC [4] makes the fit robust:

1. Randomly pick the minimal sample (two point pairs) and fit a transform.
2. Count the *inliers*: pairs that agree with it within a few pixels.
3. Repeat, keep the transform with the most inliers, and refit it on all of them.

Ordinary least squares would be pulled toward the outliers; RANSAC ignores them. The per-frame displacement is the magnitude of the translation, $d = \sqrt{t_x^2 + t_y^2}$, since the camera moves both horizontally and vertically.

6 Accumulation and Threshold

Per-frame displacements are a few pixels. The script sums them since the last saved frame and saves when the total reaches $0.5 \times 640 = 320$ pixels. At that point the view has slid about half a frame, so neighboring saved frames overlap by about half or less. Sampling therefore depends on *distance covered* rather than *elapsed time*: slow and fast walking produce similarly spaced frames, which is the main advantage over sampling one frame per second.

Two limitations apply. Small per-frame errors accumulate (*drift*), but this is harmless because the sum resets at every saved frame. And a single translation ignores *parallax*: near plants move across the image faster than far ones, so RANSAC follows whichever depth layer has the most points. That is sufficient for deciding when to sample, but it is not a metric measurement of how far the camera traveled.

7 Blur Rejection: Variance of the Laplacian

The Laplacian $\nabla^2 I = \partial^2 I/\partial x^2 + \partial^2 I/\partial y^2$ responds strongly at sharp edges. A sharp image has many strong responses of both signs and therefore a high response variance; motion blur spreads edges out and lowers it [5]. When a frame is due to be saved but scores below `MIN_SHARP` $= 50$, the script waits for the next sharp frame. The score depends on scene content, so the threshold must be tuned per dataset.

8 Parameters and Results

Parameter	Value	Meaning
`SMALL_W`, `SMALL_H`	640×360	resolution for motion estimation
`MOVE_FRAC`	0.5	save after shifting this fraction of the width
`MIN_SHARP`	50	minimum Laplacian variance

Run on all 40 videos (24 fps), the script kept 463 of 16,293 frames (about 3%): 237 from the 2-foot camera and 226 from the 4-foot camera, or 9 to 15 frames per video.

9 Core Code

```
def shift_between(prev_gray, gray):
    pts_old = cv2.goodFeaturesToTrack(prev_gray, 300, 0.01, 8)
    if pts_old is None:
        return 0.0
    pts_new, status, _ = cv2.calcOpticalFlowPyrLK(prev_gray, gray, pts_old, None)
    found = status.ravel() == 1
    if found.sum() < 10:
        return 0.0
    M, _ = cv2.estimateAffinePartial2D(pts_old[found], pts_new[found],
                                       method=cv2.RANSAC)
    return 0.0 if M is None else float(np.hypot(M[0, 2], M[1, 2]))
```

Further Reading

- R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed., free at `https://szeliski.org/Book`. Feature detection (Ch. 7), motion estimation and optical flow (Chs. 8–9).
- OpenCV tutorials, *Shi-Tomasi Corner Detector & Good Features to Track* and *Optical Flow*, `https://docs.opencv.org`.
- A. Rosebrock, "Blur detection with OpenCV," PyImageSearch, 2015.

References

[1] J. Shi and C. Tomasi, "Good features to track," in *Proc. IEEE CVPR*, 1994, pp. 593–600.

[2] B. D. Lucas and T. Kanade, "An iterative image registration technique with an application to stereo vision," in *Proc. IJCAI*, 1981, pp. 674–679.

[3] J.-Y. Bouguet, "Pyramidal implementation of the Lucas Kanade feature tracker," Intel Corporation, Microprocessor Research Labs, Tech. Rep., 2000.

[4] M. A. Fischler and R. C. Bolles, "Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.

[5] J. L. Pech-Pacheco, G. Cristóbal, J. Chamorro-Martínez, and J. Fernández-Valdivia, "Diatom autofocusing in brightfield microscopy: A comparative study," in *Proc. ICPR*, 2000, pp. 314–317.